

Padrões e Arquitetura de Software

EFC 1: Refatoração Guiada por SOLID, Clean Code e Padrões GoF

Prof. Dr. Douglas H. S. Abreu
Faculdade de Análise de Sistemas, PUC-Campinas

Maio de 2026

Resumo executivo do trabalho. Você recebeu um sistema legado funcional, porém mal projetado. O objetivo é refatorá-lo aplicando SOLID, Clean Code e ao menos quatro padrões GoF. A refatoração é guiada por testes de caracterização (Golden Master), avaliada por métricas automatizadas e validada por três extensões obrigatórias que provam OCP empiricamente. O fluxo é dividido em três sprints com checkpoints ao longo de 15 dias, seguidos de 7 dias para revisão cruzada entre grupos, e exige reflexão metacognitiva sobre uso de IA. A entrega monolítica não é aceita.

1 Objetivos de Aprendizagem

Ao concluir este trabalho, espera-se que o aluno seja capaz de:

1. Identificar violações concretas dos cinco princípios SOLID em código real.
2. Refatorar legado com rede de segurança via Golden Master Tests, sem regressão funcional.
3. Aplicar pelo menos quatro padrões GoF (Strategy, Repository, Observer e Factory são os mais naturais aqui) justificando a escolha.
4. Medir objetivamente qualidade de código via ferramentas (cobertura, complexidade ciclomática, lint, type hints).
5. Estender o sistema sem modificar código existente, evidenciando OCP de forma empírica.
6. Conduzir e receber revisão de código (peer review) com profissionalismo.
7. Refletir criticamente sobre o uso de IA generativa no processo de design.

2 Contexto e Narrativa

Você foi recém contratado como tech lead da *Loja Verde*, um e-commerce de médio porte. O sistema de pedidos funciona, mas a CTO está preocupada: a cada novo requisito do produto, mudanças simples levam dias e introduzem bugs em partes não relacionadas do código. A diretoria já anunciou três expansões para os próximos trimestres: pagamento em criptomoeda, integração com WhatsApp para notificações, e um programa de fidelidade refinado. Antes que essas mudanças cheguem, a CTO te deu duas semanas para deixar a base saudável.

Sua entrega não é só código limpo. É a **prova** de que a base agora aceita extensões com baixo custo.

3 Estrutura em Sprints

O trabalho de refatoração tem prazo total de **15 dias corridos**, dividido em três sprints com checkpoints obrigatórios. Após a entrega final do código, abre-se uma janela adicional de **7 dias corridos** exclusivamente para o Peer Review cruzado. Não há apresentação presencial; toda a avaliação é feita a partir do repositório Git, do PDF de análise e do documento de revisão cruzada.

Etapa	Foco	Peso
Sprint 0 (3 dias)	Golden Master Tests + análise textual das violações	15%
Sprint 1 (6 dias)	SRP, ISP, extração de Repository e Service	20%
Sprint 2 (6 dias)	OCP, DIP, LSP, Strategy, Observer, Factory	25%
Extensões obrigatórias	Três features novas sem modificar código existente	15%
Documento Final	Análise, diagrama UML, reflexão metacognitiva	10%
Peer Review (7 dias)	Revisão estruturada do código de outro grupo	15%

Tabela 1: Distribuição da nota por etapa.

3.1 Sprint 0: Estabelecer rede de segurança (3 dias)

Antes de mudar uma linha do código, você precisa entender o que ele faz **hoje** e capturar esse comportamento em testes.

Entregas obrigatórias:

- Suíte `pytest` com Golden Master Tests cobrindo no mínimo os fluxos: criação de pedido normal, criação de pedido VIP, criação de pedido corporativo, processamento de pagamento (cartão, PIX, boleto), atualização de status, cancelamento, geração de relatórios.
- Cobertura mínima de 80% medida por `coverage.py`.
- Documento de análise textual inicial (1 a 2 páginas) identificando ao menos 2 violações por princípio SOLID, com trecho do código, justificativa e impacto.
- Tag Git `sprint-0` marcando a entrega.

Por que começar por testes? Refatoração, por definição, é mudar estrutura sem alterar comportamento. Sem testes, você está *reescrevendo*, não refatorando. Esta é a separação técnica fundamental que esta disciplina ensina. Referências: Feathers, *Working Effectively with Legacy Code*, cap. 13; Fowler, *Refactoring*, cap. 1.

3.2 Sprint 1: SRP, ISP e separação em camadas (6 dias)

Entregas obrigatórias:

- Código organizado em camadas explícitas: `models`, `repositories`, `services`, `interfaces`.
- Pelo menos uma classe Repository (padrão **Repository**) isolando o acesso a SQLite.
- Pelo menos uma interface abstrata (ABC) por camada de colaboração externa.
- Todos os Golden Master Tests do Sprint 0 continuam passando.
- Histórico de commits com mensagens significativas (**Conventional Commits** é sugerido).
- Tag Git `sprint-1` marcando a entrega.

3.3 Sprint 2: OCP, DIP, LSP, padrões GoF e Extensões (6 dias)

Entregas obrigatórias:

- Padrão **Strategy** aplicado a descontos e métodos de pagamento.
- Padrão **Observer** ou Pub/Sub aplicado a notificações (cliente recebe email; VIP recebe email e SMS; corporativo recebe email e notifica o gerente de conta).
- Padrão **Factory Method** ou Abstract Factory para criação de pedidos.
- Injeção de dependência explícita (construtores recebem abstrações, não criam concretos).
- Hierarquia `PedEspecial` corrigida ou eliminada; se mantida, deve respeitar Liskov.
- Type hints completos verificados por `mypy -strict`.
- Diagrama UML de classes em PlantUML ou Mermaid.
- **Três extensões obrigatórias** implementadas (ver subseção a seguir).
- Tag Git `sprint-2` marcando a entrega final do código.

3.4 Extensões Obrigatórias

Após concluir a refatoração, o grupo deve implementar as três features abaixo. Cada uma deve ser entregue em commits separados, claramente datados após a refatoração principal:

1. **Pagamento em criptomoeda** com taxa de 2% sobre o valor do pedido.
2. **Canal de notificação WhatsApp** disponível para todos os tipos de cliente.
3. **Desconto progressivo por volume**: 3 ou mais unidades do mesmo item recebem 15% de desconto adicional.

Regra fundamental: cada extensão deve ser implementada **sem modificar nenhuma classe existente**, somente adicionando novas. Testes específicos para cada feature devem ser entregues junto com o código. Os Golden Master Tests originais devem continuar passando.

Estas três extensões são a prova empírica de OCP. Arquitetura ruim torna esta etapa dolorosa ou impossível; arquitetura sólida a torna trivial. Se você precisar editar uma classe existente para implementar qualquer das três features, a refatoração falhou em OCP, e a nota desta etapa é zerada para a feature correspondente. Adicionar novas classes, novas estratégias, novos observers ou novas factories é o caminho correto.

3.5 Peer Review Cruzado (7 dias após o fim do Sprint 2)

Após o fim do Sprint 2, o professor redistribui os repositórios (pseudonimizados) entre os grupos. Cada grupo tem 7 dias corridos para entregar a revisão. A entrega é um documento de 2 páginas contendo:

- Avaliação dos cinco princípios SOLID na entrega revisada (cada princípio com nota de 0 a 10 e justificativa baseada em trechos do código).
- Identificação dos padrões GoF utilizados e adequação da aplicação.
- Verificação se as três extensões obrigatórias foram implementadas sem modificar classes existentes (auditoria via `git log` e `git diff`).
- Melhor decisão de design observada, com justificativa técnica.
- Pior decisão de design observada, com justificativa.
- Uma sugestão concreta de melhoria, incluindo trecho de código alternativo.

A nota é do **revisor**, não do revisado. Aprende-se SOLID muito mais revisando do que escrevendo.

4 O Código Legado

```
1 import sqlite3
2 import json
3 from datetime import datetime
4
5
6 class Sis:
7     def __init__(self):
8         self.db = sqlite3.connect('loja.db')
9         self.c = self.db.cursor()
10        self.c.execute('''CREATE TABLE IF NOT EXISTS ped (
11            id INTEGER PRIMARY KEY, cli TEXT, itens TEXT,
12            tot REAL, st TEXT, dt TEXT, tp TEXT)''')
13        self.db.commit()
14
15    def add_ped(self, n, its, t):
16        dt = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
17        tot = 0
18        for i in its:
19            if i['tipo'] == 'normal':
20                tot += i['p'] * i['q']
21            elif i['tipo'] == 'desc10':
```

```

22         tot += i['p'] * i['q'] * 0.9
23     elif i['tipo'] == 'desc20':
24         tot += i['p'] * i['q'] * 0.8
25     elif i['tipo'] == 'frete_gratis':
26         tot += i['p'] * i['q']
27
28     if t == 'vip':
29         tot = tot * 0.95
30     elif t == 'corporativo':
31         tot = tot * 0.90
32
33     its_str = json.dumps(its)
34     self.c.execute(
35         "INSERT INTO ped (cli, itens, tot, st, dt, tp) VALUES (?, ?, ?, ?, ?, ?)",
36         (n, its_str, tot, 'pendente', dt, t))
37     self.db.commit()
38
39     # canal de notificacao depende do tipo de cliente
40     if t == 'normal':
41         print(f"Email enviado para {n}: Pedido recebido!")
42     elif t == 'vip':
43         print(f"Email enviado para {n}: Pedido recebido!")
44         print(f"SMS enviado para {n}: Pedido VIP recebido!")
45     elif t == 'corporativo':
46         print(f"Email enviado para {n}: Pedido recebido!")
47         print(f"Notificacao enviada ao gerente de conta de {n}")
48
49     return self.c.lastrowid
50
51 def get_ped(self, id):
52     self.c.execute("SELECT * FROM ped WHERE id=?", (id,))
53     r = self.c.fetchone()
54     if r:
55         return {'id': r[0], 'cli': r[1], 'itens': json.loads(r[2]),
56                 'tot': r[3], 'st': r[4], 'dt': r[5], 'tp': r[6]}
57     return None
58
59 def upd_st(self, id, s):
60     p = self.get_ped(id)
61     if p:
62         self.c.execute("UPDATE ped SET st=? WHERE id=?", (s, id))
63         self.db.commit()
64         if s == 'aprovado':
65             print(f"Email enviado para {p['cli']}: Pedido aprovado!")
66             if p['tp'] == 'vip':
67                 print(f"SMS enviado para {p['cli']}: Pedido aprovado!")
68         elif s == 'enviado':
69             print(f"Email enviado para {p['cli']}: Pedido enviado!")
70         elif s == 'entregue':
71             print(f"Email enviado para {p['cli']}: Pedido entregue!")
72             if p['tp'] == 'vip':
73                 pts = int(p['tot'] * 2)
74                 print(f"Cliente VIP ganhou {pts} pontos!")
75             elif p['tp'] == 'corporativo':
76                 pts = int(p['tot'] * 1.5)
77                 print(f"Cliente corporativo ganhou {pts} pontos!")
78             else:
79                 pts = int(p['tot'])
80                 print(f"Cliente ganhou {pts} pontos!")
81
82 def calc_tot_cli(self, n):
83     self.c.execute("SELECT * FROM ped WHERE cli=?", (n,))
84     rs = self.c.fetchall()
85     t = 0
86     for r in rs:
87         t += r[3]
88     return t
89

```

```

90 def gerar_rel(self, tipo):
91     if tipo == 'vendas':
92         self.c.execute("SELECT * FROM ped")
93         rs = self.c.fetchall()
94         print("=== RELATORIO DE VENDAS ===")
95         tot_g = 0
96         for r in rs:
97             print(f"Pedido #{r[0]} - Cliente: {r[1]} - "
98                   f"Total: R${r[3]:.2f} - Status: {r[4]}")
99             tot_g += r[3]
100        print(f"Total Geral: R${tot_g:.2f}")
101        with open('rel_vendas.txt', 'w') as f:
102            f.write(f"Total de vendas: {tot_g}")
103    elif tipo == 'clientes':
104        self.c.execute("SELECT DISTINCT cli, tp FROM ped")
105        rs = self.c.fetchall()
106        print("=== RELATORIO DE CLIENTES ===")
107        for r in rs:
108            n = r[0]
109            tp = r[1]
110            tot = self.calc_tot_cli(n)
111            print(f"Cliente: {n} ({tp}) - Total gasto: R${tot:.2f}")
112            with open('rel_clientes.txt', 'w') as f:
113                for r in rs:
114                    f.write(f"{r[0]},{r[1]}\n")
115
116 def proc_pag(self, id, m, vl):
117     p = self.get_ped(id)
118     if not p:
119         return False
120     if vl < p['tot']:
121         print("Valor insuficiente!")
122         return False
123     if m == 'cartao':
124         print("Processando pagamento com cartao...")
125         print("Cartao validado!")
126         self.upd_st(id, 'aprovado')
127         return True
128     elif m == 'pix':
129         print("Gerando QR Code PIX...")
130         print("PIX recebido!")
131         self.upd_st(id, 'aprovado')
132         return True
133     elif m == 'boleto':
134         print("Gerando boleto...")
135         print("Boleto gerado!")
136         return True
137     else:
138         print("Metodo de pagamento invalido!")
139         return False
140
141 def validar_estoque(self, its):
142     # TODO: integrar com sistema de estoque externo
143     est = {'produto1': 100, 'produto2': 50, 'produto3': 75}
144     for i in its:
145         if i['nome'] not in est:
146             print(f"Produto {i['nome']} nao encontrado!")
147             return False
148         if est[i['nome']] < i['q']:
149             print(f"Estoque insuficiente para {i['nome']}!")
150             return False
151     return True
152
153 def cancelar_pedido(self, id):
154     # cancela sem validar regras de negocio
155     self.c.execute("UPDATE ped SET st=? WHERE id=?", ('cancelado', id))
156     self.db.commit()
157     print(f"Pedido {id} cancelado")

```

```

158
159     def close(self):
160         self.db.close()
161
162
163 class PedEspecial(Sis):
164     def add_ped(self, n, its, t):
165         dt = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
166         tot = 0
167         for i in its:
168             if i['tipo'] == 'normal':
169                 tot += i['p'] * i['q']
170             elif i['tipo'] == 'desc10':
171                 tot += i['p'] * i['q'] * 0.9
172             elif i['tipo'] == 'desc20':
173                 tot += i['p'] * i['q'] * 0.8
174         tot = tot * 1.15
175         its_str = json.dumps(its)
176         self.c.execute(
177             "INSERT INTO ped (cli, itens, tot, st, dt, tp) VALUES (?, ?, ?, ?, ?, ?)",
178             (n, its_str, tot, 'pendente', dt, t))
179         self.db.commit()
180         print(f"Email especial enviado para {n}: Pedido especial recebido!")
181         return self.c.lastrowid
182
183     def upd_st(self, id, s):
184         # PedEspecial pula direto para qualquer estado
185         # ignorando transicoes intermediarias do pai
186         p = self.get_ped(id)
187         if p:
188             self.c.execute("UPDATE ped SET st=? WHERE id=?", (s, id))
189             self.db.commit()
190             print(f"Pedido especial {id} -> {s}")
191
192
193 def main():
194     s = Sis()
195     its1 = [
196         {'nome': 'produto1', 'p': 100, 'q': 2, 'tipo': 'normal'},
197         {'nome': 'produto2', 'p': 50, 'q': 1, 'tipo': 'desc10'}
198     ]
199     if s.validar_estoque(its1):
200         id1 = s.add_ped('Joao Silva', its1, 'normal')
201         print(f"Pedido {id1} criado!")
202         s.proc_pag(id1, 'cartao', 250)
203         s.upd_st(id1, 'enviado')
204         s.upd_st(id1, 'entregue')
205
206     its2 = [{'nome': 'produto3', 'p': 200, 'q': 1, 'tipo': 'desc20'}]
207     if s.validar_estoque(its2):
208         id2 = s.add_ped('Maria Santos', its2, 'vip')
209         s.proc_pag(id2, 'pix', 160)
210
211     its3 = [{'nome': 'produto1', 'p': 100, 'q': 5, 'tipo': 'normal'}]
212     if s.validar_estoque(its3):
213         id3 = s.add_ped('Empresa XYZ', its3, 'corporativo')
214         s.proc_pag(id3, 'boleto', 500)
215
216     s.gerar_rel('vendas')
217     print()
218     s.gerar_rel('clientes')
219     s.close()
220
221
222 if __name__ == '__main__':
223     main()

```

5 Padrões GoF Exigidos

A disciplina é *Padrões e Arquitetura*. Não basta refatorar; é preciso aplicar e **nomear** no documento de análise pelo menos quatro padrões da seguinte lista:

Padrão	Aplicação natural no exercício
Strategy	Algoritmos de desconto e métodos de pagamento intercambiáveis.
Repository	Isolamento do acesso a SQLite, permitindo trocar persistência.
Observer / Pub-Sub	Notificações desacopladas da regra de negócio principal.
Factory Method	Criação de Pedidos diferenciados por tipo de cliente.
Template Method	Esqueleto comum do fluxo de processamento de pagamento.
Chain of Responsibility	Cadeia de validações (estoque, pagamento, cliente).

Tabela 2: Padrões GoF aplicáveis e onde encaixam.

Para cada padrão usado, o documento deve conter: nome do padrão, intenção segundo GoF, classes e interfaces criadas, justificativa de aplicação no contexto, e diagrama parcial.

6 Métricas Automatizadas

Parte da nota é objetiva e verificável via CI. Configure as ferramentas abaixo e inclua o relatório no PDF final.

Métrica	Ferramenta	Critério mínimo
Cobertura de testes	<code>coverage.py</code>	$\geq 85\%$
Lint / PEP 8	<code>ruff</code> ou <code>pylint</code>	$\text{score} \geq 8.5$
Complexidade ciclomática	<code>radon cc</code>	nota máxima B ($CC \leq 10$)
Linhas por método	<code>radon raw</code> ou inspeção	≤ 20
Type hints	<code>mypy -strict</code>	sem erros
Métodos públicos por classe	inspeção	≤ 7 (cheiro de SRP)

Tabela 3: Métricas objetivas obrigatórias.

Sugere-se um Makefile simples para automatizar:

```
.PHONY: test lint type cov complexity all

test:
    pytest -v

cov:
    pytest --cov=. --cov-report=term-missing --cov-report=html

lint:
    ruff check .

type:
    mypy --strict src/

complexity:
    radon cc src/ -s -a

all: lint type test cov complexity
```

7 Exemplo de Golden Master Test

Para acelerar o Sprint 0, segue um teste inicial sugerido. Você deve estendê-lo, não apenas reproduzi-lo.

```

1 import pytest
2 from legacy import Sis
3
4
5 @pytest.fixture
6 def sis(tmp_path, monkeypatch):
7     """Isola o banco em diretorio temporario por teste."""
8     monkeypatch.chdir(tmp_path)
9     s = Sis()
10    yield s
11    s.close()
12
13
14 def test_pedido_normal_calcula_total_corretamente(sis):
15     itens = [
16         {'nome': 'produto1', 'p': 100, 'q': 2, 'tipo': 'normal'},
17         {'nome': 'produto2', 'p': 50, 'q': 1, 'tipo': 'desc10'},
18     ]
19     id_ped = sis.add_ped('Joao Silva', itens, 'normal')
20     pedido = sis.get_ped(id_ped)
21     assert pedido['tot'] == pytest.approx(245.0)
22     assert pedido['st'] == 'pendente'
23
24
25 def test_pedido_vip_aplica_desconto_de_5_por_cento(sis):
26     itens = [{'nome': 'p1', 'p': 100, 'q': 1, 'tipo': 'normal'}]
27     id_ped = sis.add_ped('Maria', itens, 'vip')
28     pedido = sis.get_ped(id_ped)
29     assert pedido['tot'] == pytest.approx(95.0)
30
31
32 def test_pagamento_insuficiente_falha(sis):
33     itens = [{'nome': 'p1', 'p': 100, 'q': 1, 'tipo': 'normal'}]
34     id_ped = sis.add_ped('Joao', itens, 'normal')
35     assert sis.proc_pag(id_ped, 'cartao', 50) is False
36
37
38 def test_pix_aprova_pedido_automaticamente(sis):
39     itens = [{'nome': 'p1', 'p': 100, 'q': 1, 'tipo': 'normal'}]
40     id_ped = sis.add_ped('Joao', itens, 'normal')
41     sis.proc_pag(id_ped, 'pix', 100)
42     assert sis.get_ped(id_ped)['st'] == 'aprovado'
43
44
45 def test_boleto_nao_aprova_automaticamente(sis):
46     itens = [{'nome': 'p1', 'p': 100, 'q': 1, 'tipo': 'normal'}]
47     id_ped = sis.add_ped('Joao', itens, 'normal')
48     sis.proc_pag(id_ped, 'boleto', 100)
49     assert sis.get_ped(id_ped)['st'] == 'pendente'

```

8 Documento de Análise

O PDF final deve conter as seguintes seções, nesta ordem:

8.1 (a) Identificação das Violações SOLID

Para **cada** um dos cinco princípios, descreva ao menos duas violações:

- Trecho de código problemático (citar arquivo e linhas).
- Princípio violado e justificativa técnica.
- Impacto prático: que tipo de mudança futura ficaria cara?

8.2 (b) Soluções Implementadas

Para cada violação:

- Solução adotada e princípio que ela aplica.
- Padrão GoF utilizado (se aplicável).
- Classes e interfaces criadas.
- Trecho do código antes e depois.

8.3 (c) Diagrama UML de Classes

Diagrama em PlantUML ou Mermaid mostrando a arquitetura final, com camadas explícitas e setas de dependência respeitando DIP.

8.4 (d) Melhorias de Clean Code

Liste e justifique: refatorações de nomenclatura, métodos extraídos, abstrações criadas, eliminação de duplicação.

8.5 (e) Extensões Implementadas

Documentar as três extensões obrigatórias, com:

- Diff conceitual de cada feature (que arquivos foram adicionados, nenhum modificado).
- Trecho de código de cada extensão.
- Justificativa de por que a refatoração tornou cada extensão simples.

8.6 (f) Reflexão Metacognitiva

Duas subseções curtas (máximo 1 página somando as duas):

- **O princípio que ainda não dominei.** Escolha um dos cinco princípios SOLID e explique honestamente onde você ainda tem dúvidas. Esta seção **não** reduz nota; respostas vazias ou óbvias sim.
- **Como usei IA neste trabalho.** Quais prompts funcionaram, quais falharam, e cite ao menos uma decisão de design onde você **discordou** da sugestão da IA, justificando.

O uso de IA não é proibido e nem desencorajado neste trabalho. O que estamos avaliando é a sua capacidade de *dirigir* a ferramenta com critério técnico, não a sua capacidade de copiar a saída dela. Trabalhos cuja análise textual contradiz o próprio código entregue serão penalizados independentemente da origem do código.

9 Estrutura Sugerida de Projeto

```
projeto/
|-- src/
|   |-- main.py
|   |-- models/
|   |   |-- order.py
|   |   |-- customer.py
|   |   '-- order_item.py
|   |-- repositories/
|   |   |-- order_repository.py
|   |   '-- interfaces.py
|   |-- services/
|   |   |-- order_service.py
|   |   |-- payment_service.py
|   |   '-- notification_service.py
|   |-- strategies/
```

```

|   |   |-- discount_strategy.py
|   |   '-- payment_strategy.py
|   '-- observers/
|       '-- notification_observer.py
|-- tests/
|   |-- golden_master/
|   |   '-- test_legacy_behavior.py
|   |-- unit/
|   '-- integration/
|-- legacy.py
|-- Makefile
|-- pyproject.toml
|-- README.md
'-- docs/
    |-- analise.md
    '-- diagrama.puml

```

A estrutura é sugestão. Outras organizações são aceitas desde que justificadas.

10 Rubrica Detalhada

Critério (código, 60 pts)	Pontos
Golden Master Tests passando após refatoração	10
SRP e ISP (estrutura em camadas correta)	10
OCP e DIP (Strategy + injeção de dependências)	10
LSP (hierarquia correta ou eliminação justificada)	5
Padrões GoF nomeados e justificados (mínimo 4)	10
Métricas automatizadas (lint, coverage, complexidade)	10
Commits incrementais com mensagens significativas	5
Critério (extensões obrigatórias, 15 pts)	Pontos
Pagamento em criptomoeda sem modificar código antigo	5
Notificação WhatsApp sem modificar código antigo	5
Desconto por volume sem modificar código antigo	5
Critério (análise, 10 pts)	Pontos
Identificação precisa das violações SOLID	4
Diagrama de classes (UML PlantUML ou Mermaid)	2
Reflexão metacognitiva e uso transparente de IA	2
Apresentação, organização e citações	2
Critério (peer review, 15 pts)	Pontos
Avaliação SOLID estruturada do código revisado	6
Identificação correta dos padrões GoF do grupo revisado	3
Auditoria das três extensões (foram aditivas?)	3
Sugestão concreta de melhoria com código alternativo	3
Total	100

Tabela 4: Rubrica detalhada de avaliação.

11 Cronograma

Data	Marco	Entrega
11/05/2026	Liberação do trabalho	Repositório criado
14/05/2026	Fim do Sprint 0	Golden Master + análise textual inicial
20/05/2026	Fim do Sprint 1	SRP, ISP, Repository
26/05/2026	Fim do Sprint 2 + Extensões	Código final, padrões GoF, três features novas, PDF
02/06/2026	Fim do Peer Review	Revisão do código de outro grupo entregue

Tabela 5: Cronograma: 15 dias para execução, 7 dias para peer review.

12 Observações Importantes

- A funcionalidade original **não** pode ser removida.
- Novas features podem ser adicionadas se justificadas.
- O repositório Git é parte da avaliação: commits grandes e únicos no final perdem pontos.
- Trabalhos em que a análise textual contradiz o código entregue serão penalizados.
- As tags **sprint-0**, **sprint-1** e **sprint-2** devem ser criadas nas datas indicadas no cronograma. Tags retroativas serão detectadas pelo **git reflog** e desconsideradas.

13 Formato de Entrega

- **Repositório Git** no GitHub Classroom com todas as etapas e tags.
- **PDF final** contendo: relatório de análise, diagrama UML, reflexão metacognitiva, saída dos testes e relatório de métricas.
- **Nome do arquivo:** grupoXX_pas_efc1.pdf
- **Prazo de entrega final do código:** 26/05/2026, 22:40.
- **Prazo de entrega do peer review:** 02/06/2026, 22:40.

14 Leituras Recomendadas

1. Martin, R. C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008. Capítulos 2, 3, 6, 10.
2. Feathers, M. *Working Effectively with Legacy Code*. Prentice Hall, 2004. Capítulos 13 (Golden Master) e 22.
3. Gamma, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. Strategy, Observer, Factory Method.
4. Fowler, M. *Refactoring: Improving the Design of Existing Code*. 2ª ed. Addison-Wesley, 2018.
5. North, D. *CUPID for joyful coding*. Disponível em <https://dannorth.net/cupid-for-joyful-coding/>.

Lembre-se: O objetivo não é apenas fazer o código funcionar, mas criar um sistema que aceite mudança sem dor. As três extensões obrigatórias são a hora em que esse design é cobrado empiricamente. Pense como um tech lead que precisa entregar software de qualidade e antecipar requisitos futuros.

Bom trabalho!